

Library Management System - Design Patterns Report

BIM492 - Design Patterns Project

1. Overview

This document details the architectural decisions made in the Library Management System. To ensure a robust, maintainable, and extensible architecture, three core Design Patterns were integrated into the system: Singleton, Strategy, and Command patterns. These patterns directly shape the runtime behavior of the system and allow for varying behaviors without modifying existing core classes.

2. Singleton Pattern

The Singleton Pattern is used to restrict the instantiation of a class to a single object. In our system, the `LibraryManager` class is implemented as a Singleton.

- **Implementation Details:**

`LibraryManager` has a private constructor and a static `getInstance()` method. This guarantees that there is only one central coordinator managing the inventory of books, the list of registered members, and the global command history.

- **Runtime Behavior Impact:**

By ensuring a single instance, the system prevents data inconsistencies that would arise if multiple managers existed. Whether a book is added, modified, searched, or borrowed, all UI flows access the exact same state in memory.

3. Strategy Pattern

The Strategy Pattern is used to define a family of algorithms, encapsulate each one, and make them interchangeable. It lets the algorithm vary independently from clients that use it.

- **Implementation Details:**

Two strategy interfaces were created: `ISearchStrategy` and `ISortStrategy`. Concrete classes (e.g., `SearchByTitle`, `SearchByAuthor`, `SearchByIsbn`, `SearchByCategory`, `SearchByTag`) implement the `ISearchStrategy` interface. Similarly, `TitleAscendingSortStrategy` implements `ISortStrategy`.

- **Runtime Behavior Impact:**

Library Management System - Design Patterns Report

BIM492 - Design Patterns Project

When the user selects a specific search criteria from the CLI, the LibraryManager's internal search strategy is dynamically swapped at runtime. The core search execution code in LibraryManager remains completely agnostic to the specific algorithm being executed. This perfectly satisfies the Open/Closed Principle, as adding a new search type requires no modification to existing core classes.

4. Command Pattern

The Command Pattern encapsulates a request as an object, thereby allowing us to parameterize clients with different requests, queue or log requests, and support undoable operations.

- **Implementation Details:**

The ICommand interface defines execute() and undo() methods. The ModifyBookCommand implements this interface. When a book is modified, a deep copy of the original book (backupBook) and the new state (newData) are stored inside the ModifyBookCommand object. These command objects are then pushed into a CommandHistory stack.

- **Runtime Behavior Impact:**

The system's modification workflow operates by executing these encapsulated command objects. If the user selects the 'Undo Last Modification' option, the CommandHistory pops the most recent command and invokes its undo() method, which securely restores the target book's state using the backup. This shapes the runtime flow by detaching the action definition from its execution and enables a robust LIFO undo functionality without bloating the Book or LibraryManager classes.